

T03 - Fleet & Scenario Data Engine

Completion report | Fleet Transition Planner (CSD3401 / GAM300) | 22 September 2026

Deliverable	T03 - Fleet & Scenario Data Engine (M1 contract: required)
Owner	Tan Wei Jun
Supports	F01, F02, F03, F04, F05, F07
Branch	Fleet-and-Scenario-Data-Engine (not pushed; main untouched)
Commits	0a385dd - T03: Fleet & Scenario Data Engine 338945a - T03: prove fleet and plans survive save/reopen
Change size	38 files changed, 2,034 insertions, 633 deletions
Verification	pnpm verify passes - 141 client tests, 11 UI, 1 server; typecheck and all three builds green

Summary

T03 replaces the mock fleet with the canonical domain engine for fleet vehicles, preset references and scenario transition plans. The single transition rule - a vehicle runs on its current preset before its transition year and on the scenario's target preset from that year onward - now lives in exactly one module, and the 3D preview, timeline and comparison preview all read it instead of each keeping their own copy.

Presets were also brought up to the shape docs/tech/contracts.md specifies (range and full economics), the preset seed de/serializer was removed at the owner's instruction, and project documents now persist at version 3 with fleet and analysis settings included.

1. Responsibilities audit

Each row is a responsibility as written in docs/tech-tasks/T03 - Fleet and Scenario Data Engine.md.

#	Responsibility (verbatim)	Status	Where implemented
1	Define/own the M1 domain contracts for FleetVehicle, preset references and vehicle transition plans	Done	domain/contracts.ts, vehicles/types.ts, domain/fleet.ts
2	Replace MockVehicle as the authoritative fleet source	Done	state/fleetStore.ts rewritten; MockVehicle and initialVehicles deleted
3	Provide real fleet CRUD/domain operations and stable vehicle IDs	Done	fleetStore: create / update / duplicate / remove; ids immutable under patch
4	Validate vehicle -> preset and scenario -> vehicle/preset references	Done	domain/references.ts; enforced on write in fleetStore and projectStore
5	Keep transition year/target preset scenario-owned so editing one Scenario does not mutate another	Done	domain/scenario.ts deep-copy; proven in fleetCommands.test.ts
6	Resolve the effective vehicle/preset state for a requested scenario and selected year	Done	domain/effectiveState.ts
7	Provide deterministic snapshots/events consumed by T04, T05 and the 3D/product views	Done	transitionEvents(), effectiveFleetState(), currentSimulationInput()
8	Expose serializable authoritative inputs to T01/T06; do not own browser persistence itself	Done	domain/projectDocument.ts (pure); storage stays in T01's repository

Goal and M1 evidence

Requirement	Status	Evidence
Goal: canonical domain/state engine replacing mock fleet state, giving every consumer the same authoritative scenario data	Met	All four former MockVehicle consumers now read T03
M1 evidence: create/edit a real fleet vehicle, assign different target presets/transition years in two Scenarios, switch Scenario/year, and prove the effective state changes correctly without mutating the base fleet or the other Scenario	Met	fleetCommands.test.ts - "M1 evidence: two scenarios over one fleet"
Merge gate: stable vehicle/preset references across save/reopen	Met	fleetCommands.test.ts - "save and reopen" (added after audit; see section 6)

2. Files added

Eight source modules and five test files, all under `apps/client/src/domain/`.

Engine modules

File	LOC	What it does
<code>domain/effectiveState.ts</code>	98	The single transition rule. Answers what preset a vehicle runs in a given year under a given plan, and projects a scenario's plans into ordered timeline events. Refuses to invent a target the scenario never named, and never emits an event outside the analysis period.
<code>domain/fleet.ts</code>	138	Validation and construction for FleetVehicle. Enforces the contract ranges: nonnegative distances, utilisation 0-1, whole operating days up to 366, dwell hours up to 24, per-kind holding terms, and current preset resolving inside the project. Also builds blank vehicles and unique names.
<code>domain/references.ts</code>	102	Reference integrity. Lists every fleet and scenario use of a preset, every scenario plan keyed by a vehicle, removes a plan entry without mutating the source, and reports all invariant violations in a project as readable messages.
<code>domain/scenario.ts</code>	88	Scenario document handling: version 2 documents with vehicle plans plus charging assumptions, deep-copy helpers so duplication is genuinely independent, and the legacy version 1 upgrade.
<code>domain/projectDocument.ts</code>	75	Project document version 3 serialization for T01/T06, plus reading legacy version 2. Legacy presets gain deliberately neutral values so a reopened old project gains no invented costs. Unreadable records are dropped, not fatal.
<code>domain/fleetCommands.ts</code>	71	Edits that span stores: deleting a vehicle clears its plan entries in every scenario of every world first, then removes the vehicle; deleting a preset is refused while anything references it. Also assembles the SimulationInput pair for T05.
<code>domain/mockProject.ts</code>	123	Mock presets, fleet and analysis settings seeding a project that has not been saved yet. Replaces the deleted defaults.json. Every record is validated on the way out.

Test files

File	LOC	Covers
<code>domain/fleetCommands.test.ts</code>	248	Fleet CRUD, duplication independence, invalid-patch rejection, immutable ids, vehicle-delete cascade, blocked preset delete, the two-scenario M1 evidence case, save/reopen round trip, and the T05 input snapshot.
<code>domain/fleet.test.ts</code>	134	Every validation range and rejection case for vehicles and analysis settings, inclusive period bounds, and seed integrity (including that daily distance x operating days reproduces annual distance).
<code>domain/effectiveState.test.ts</code>	105	Before / at / after the transition year, absent and dangling plans, event ordering and endpoints, out-of-period suppression, and two-scenario isolation.
<code>domain/projectDocument.test.ts</code>	101	Version 3 round trip, defensive copying, version 2 and version 1 upgrades, dropped duplicates and malformed records, and malformed input not throwing.
<code>domain/references.test.ts</code>	74	Preset and vehicle reference lookups, non-mutating removal, and every invariant message.

3. Files edited

Rewritten or substantially changed

File	+/-	Change
state/fleetStore.ts	+101/-33	Rewritten. Was six hard-coded vans in a four-field MockVehicle type. Now holds real FleetVehicle records plus the project analysis settings, with create / update / duplicate / remove and an edit-baseline lifecycle. Invalid patches are rejected whole so a bad draft never replaces a good value. Selection state was deliberately left out - the contract classes selection as editor-only.
components/FleetManagementPanel.tsx	+80/-32	Rewritten. Was a read-only list with two dropdowns over mock data. Now adds, edits (annual distance, current preset), and deletes vehicles, with a confirmation that names the scenario plans the deletion will remove. Target preset and transition year are written through the validated scenario action. Years come from the project period.
components/VehiclePresents.tsx	+73/-81	Added form fields for range, yearly maintenance, charging efficiency and owned/leased acquisition terms. Removed the dev-only Save button and the library Import/Export buttons. Deletion now routes through the T03 guard and lists the references blocking it.
project/comparisonModel.ts	+47/-57	Retyped from MockVehicle to FleetVehicle and re-pointed at the shared transition rule instead of its own year arithmetic. Prices and emissions factors now come from the project analysis settings rather than local constants; maintenance cost is included in annual operating cost.
state/projectStore.ts	+94/-39	Saving, loading, exporting and dirty-state now cover fleet and analysis settings, not just presets. Scenario plan writes are validated against T03's reference rules. Added removeVehiclePlans() for the delete cascade, and scenario documents are upgraded to version 2 at the persistence boundary.
components/TimelineControl.tsx	+20/-14	Vehicle markers are now the scenario's real projected transitions rather than a schedule baked into the mock vehicles. Start and end years come from the project analysis settings.
App.tsx	+17/-12	The 3D fleet preview asks effectiveVehicleState() which preset applies in the selected year instead of recomputing the rule inline.

Smaller supporting edits

File	+/-	Change
vehicles/types.ts	+40/-11	VehiclePreset gained maintenanceCostPerYear, rangeKm, chargingEfficiency and the owned/leased acquisition union, with validation for each. Removed presetFileVersion and PresetFile.
domain/contracts.ts	+8/-14	There were two competing preset definitions. M1VehiclePreset now aliases the single VehiclePreset so the contract and the validator cannot drift apart.
state/presetStore.ts	+9/-38	Removed exportPresets / importPresets; seeds from createMockPresets(); blank presets carry neutral M1 values.
project/portableProject.ts	+15/-4	Export/import accepts project version 3 and scenario version 2 while keeping version 2 / 1 importable. Record-level validation is delegated to T03 so one bad preset no longer makes a whole file unimportable.
project/sampleProjects.ts	+21/-14	Sample project moved to version 3 with a real fleet; both sample plans now carry explicit transition decisions so the demo still shows transitions once invented targets were removed.
project/types.ts	+10/-3	Added WorkspaceScenario, so in-memory scenarios are statically the M1 shape while stored records may still be legacy.
project/analysisPeriod.ts	+13/-3	START_YEAR / END_YEAR derive from defaultAnalysisSettings instead of being independently hard-coded.
vite.config.ts	+1/-4	Dropped the savePresets dev plugin registration.

Tests updated

App.test.tsx, ProjectControls.test.tsx, TimelineControl.test.tsx, comparisonModel.test.ts, portableProject.test.ts, presetStore.test.ts, projectStore.test.ts, vehicles/types.test.ts - moved onto the new inputs shape and the new preset fields. The timeline test now asserts the stricter rule that a year alone is not a transition without a target preset.

4. Files deleted

Removed at the owner's instruction: the preset seed file and the de/serializer around it.

File	LOC	Why
vehicles/defaults.json	65	Seed preset library. Replaced by domain/mockProject.ts; saved projects carry their own presets.
vehicles/defaults.ts	26	Loader for the seed file.
vite/savePresets.ts	93	Dev-only POST /__save-presets endpoint that let the running browser rewrite the seed file on disk.

The preset library Export / Import buttons and the PresetFile format were removed with them. Project-level .fleetproject export/import is unaffected and still works.

5. Behaviour changes a reviewer will notice

Change	Detail
Fleet data now persists	Previously the six vans were a constant that reappeared on reload, so there was nothing to save. Adding or editing a vehicle now survives save and reopen with its preset reference intact.
Nothing is pre-scheduled	Most likely to surprise a teammate. The old defaultTargetPresetId invented an electric target for every vehicle, so the whole fleet appeared to have a transition plan nobody made. A scenario that names no target now plans nothing. The sample project was given explicit plans so the demo still shows transitions.
Deletion states its consequences	Deleting a preset in use is refused with the list of vehicles and scenarios to reassign first. Deleting a vehicle warns which scenario plans go with it, then removes both.
Invalid edits are rejected whole	A patch that would make a vehicle invalid leaves the previous valid record in place, per the contract's rule that invalid drafts never replace valid domain values.
Preset form is larger	Four new fields appear per preset; the Save / Import / Export buttons are gone.

6. Audit findings, gaps and boundaries

Found and fixed during this audit

No test proved the fleet survived save/reopen. The integration contract's merge gate requires stable vehicle/preset references across save and reopen. The implementation was correct, but nothing asserted it - projectStore.test.ts exercised save/open without ever checking the fleet. Two round-trip tests were added in commit 338945a covering edited vehicles, newly added vehicles, analysis settings, scenario plans, and a deleted vehicle staying deleted. Both passed on the first run, confirming an evidence gap rather than a defect.

Open items inside T03

Item	Assessment
findReferenceIssues() is exported and fully tested but not yet called from product code	Intentional. It is the whole-project invariant checker; toM1ProjectDocument() already drops bad records on load, so there is currently no code path that can produce a violation. It is ready for T01/T06 to call at a persistence boundary.
duplicateVehicle() has no UI entry point	The operation and its coverage exist. F02 owns whether the fleet panel offers a Duplicate button.

Outside T03 - other owners

Item	Owner	Note
Fleet panel only exposes annual distance and current preset for editing	F02 (Jarrel Tay Wee Han)	The domain supports all thirteen FleetVehicle fields; the remaining form controls are F02's scope.

Item	Owner	Note
No UI for analysis settings (period, currency, fuel price, emissions factors)	F04 (Elijah Chua Jye Kang)	The store action <code>updateAnalysis()</code> exists and clamps stranded replacement years.
No UI for scenario charging assumptions	F04	Scenario documents carry them at version 2 with defaults (depot charging, 0.30/kWh).
<code>timelineStore.ts</code> still uses fixed <code>START_YEAR / END_YEAR</code>	T04 (Jarrel Tay Wee Han)	The contract assigns year clamping and playback to T04. The constants now derive from <code>defaultAnalysisSettings</code> so there is a single source of the numbers, and the panels already read the live period.
Depot 3D objects carry no fleet identity	F05 / depot editor (Chew Shee Yang)	Per <code>depot-editor.md</code> , vehicles are meant to reach the depot via per-scenario bay assignment, which is documented as future scope. T03 would own the assignment records when it lands.

Shared files touched outside T03's column

The M1 integration contract asks owners to request small composition changes rather than restructure shared files. Two were edited directly, with the repository's approval, because replacing `MockVehicle` and persisting the fleet is not possible otherwise: **`state/projectStore.ts`** (T06, Brandon Koh Kai Yang) for version 3 save/load and the plan-removal cascade, and **`project/portableProject.ts`** (T01, Chew Shee Yang) for version 3 export/import validation, which previously hard-rejected anything but version 2. Both owners should review these before merge.

7. Verification

Check	Result
pnpm verify (typecheck + tests + builds, all packages)	Pass
apps/client tests	141 passed / 18 files
packages/ui tests	11 passed / 2 files
apps/server tests	1 passed / 1 file
TypeScript	tsc -b clean, strict with <code>noUnusedLocals</code> and <code>noUnusedParameters</code>
Builds	ui, server and client production builds all succeed

Note: the single apps/server test can time out during a parallel pnpm verify run purely from resource contention with the jsdom client suite. It passes in isolation in under a second, and a repeat verify run passes cleanly. This is a known flake, not a regression from this work.

Generated 22 September 2026 from branch Fleet-and-Scenario-Data-Engine at commit 338945a. Nothing has been pushed and main remains at 7b6aa28.